

Reading Material 2:

C Data Types and Type Conversions

(Learn by reading code: primitive types, modifiers, and conversions)

1. Fundamental Types in C

C has a small set of primitive data types, with **modifiers** for size and sign.

- **Integer types:** `int`, `short`, `long`, `long long`
- **Character type:** `char` (can be signed or unsigned)
- **Floating-point types:** `float`, `double`, `long double`
- **Qualifiers:** `signed`, `unsigned`
- **Special:** `void` (no value / function returns nothing)

```
#include <stdio.h>

int main() {
    char c = 'A';    // character
    int i = 42;      // typical int
    unsigned int u = 42; // only positive
    float f = 3.14f;  // single precision
    double d = 2.71828; // double precision

    printf("%c %d %u %.2f %.5f\n", c, i, u, f, d);
}
```

Takeaway: C types are simple, but vary in **size** depending on platform. Use `<limits.h>` and `<float.h>` to know exact ranges.

2. Sizes and Ranges

```
#include <stdio.h>
#include <limits.h>
#include <float.h>

int main() {
    printf("int: %zu bytes, %d to %d\n", sizeof(int), INT_MIN, INT_MAX);
}
```

```
printf("unsigned int: %zu bytes, 0 to %u\n", sizeof(unsigned int), UINT_MAX);
printf("float: %zu bytes, %.3e to %.3e\n", sizeof(float), FLT_MIN, FLT_MAX);
}
```

Typical results (on 32/64-bit GCC):

- `int`: 4 bytes, $-2,147,483,648$ to $+2,147,483,647$
- `unsigned int`: 0 to $4,294,967,295$
- `float`: 4 bytes, $\sim 10^{-38}$ to 10^{38}
- `double`: 8 bytes, $\sim 10^{-308}$ to 10^{308}

3. Integer Promotions (Automatic widening)

When doing arithmetic, smaller integer types (like `char`, `short`) are *promoted* to `int`.

```
char a = 100, b = 28;
char sum = a + b; // a and b promoted to int first
printf("%d\n", sum); // prints 128 (but stored in char, may overflow!)
```

Takeaway: Arithmetic on `char/short` is promoted to `int`. Storing result back into smaller type may cause overflow.

4. Implicit Type Conversions ("Usual Arithmetic Conversions")

When you mix types, C converts them automatically ("type promotion") to a common type.

```
int i = 5;
double d = 2.5;
printf("%f\n", i + d); // i promoted to double → result is 7.5
```

Rule of Thumb:

- Smaller $\rightarrow$ larger type.
- Integer $\rightarrow$ floating if mixed.
- Unsigned can cause surprises if mixed with signed.

5. Explicit Type Casting (Forcing conversions)

```
int i = 7, j = 2;
double ratio = (double) i / j; // cast ensures floating division
printf("%f\n", ratio);        // prints 3.500000
```

Without cast, `i / j` would be **integer division** $\rightarrow 3$.

6. Unsigned vs Signed Mixing

```
int a = -5;
unsigned int b = 2;
if (a < b)
    printf("a is less\n");
else
    printf("a is not less\n");
```

On many systems this prints “**a is not less**” because `a` is converted to unsigned $\rightarrow$ huge positive number.

Takeaway: Avoid mixing signed and unsigned types casually.

7. Floating Conversions

```
float f = 1.23456789f; // 7 decimal digits precision
double d = f;          // no extra precision beyond what float stored
long double ld = d;     // may have 18+ digits precision
```

Takeaway: Converting up (float $\rightarrow$ double $\rightarrow$ long double) is safe.
Converting down (double $\rightarrow$ float) may lose precision.

8. Practical Examples

Example 1: Division

```
int a = 5, b = 2;
printf("%d\n", a / b);    // 2 (integer division)
printf("%f\n", (double)a / b); // 2.5 (forced floating-point)
```

Example 2: Overflow

```
unsigned char x = 255;
x = x + 1;    // wraps to 0
printf("%u\n", x);
```

Example 3: Mixing

```
int i = 1000000;
float f = 0.1f;
printf("%f\n", i * f); // may lose precision due to float
```

9. Tips

- Use `sizeof` to confirm type sizes on your system.
 - Use `long long` if you need guaranteed 64-bit integer.
 - Prefer `double` over `float` unless memory/speed matters.
 - Beware of **implicit unsigned conversions**.
 - Use explicit casts when clarity matters.
-